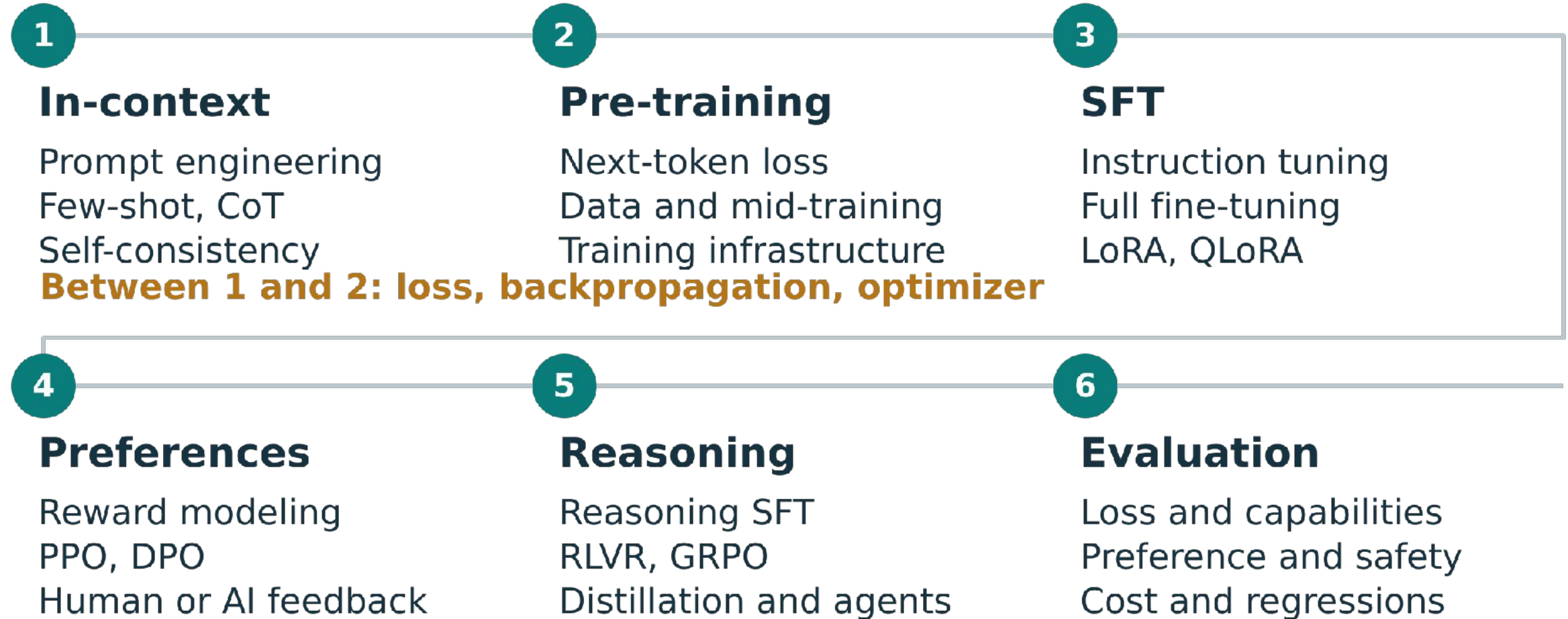


LECTURE 3

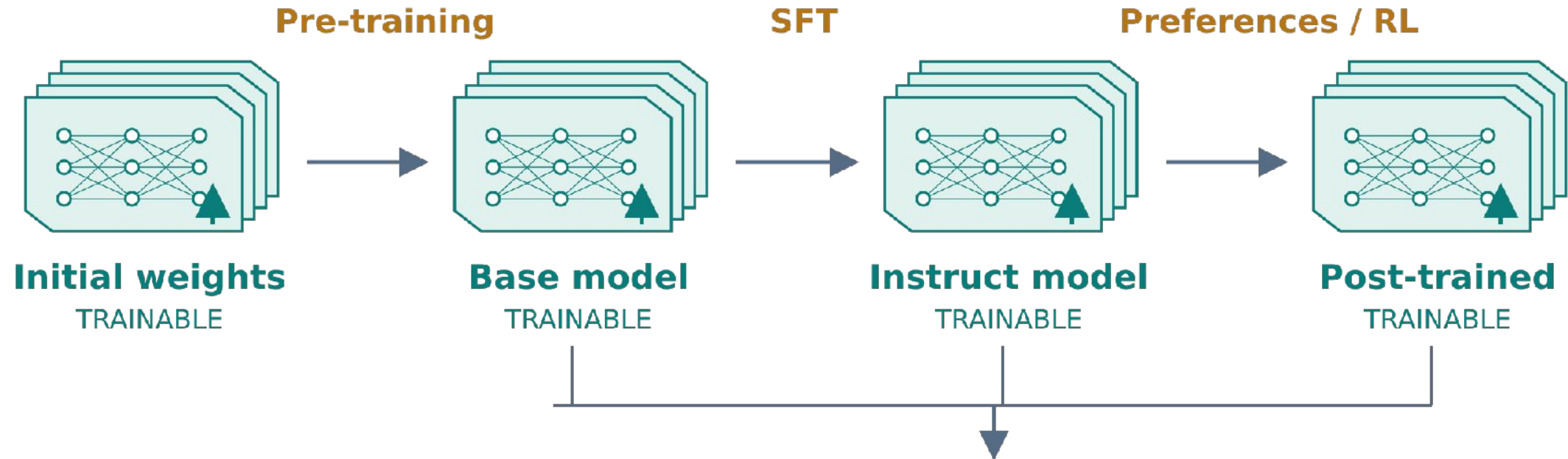
Training and Learning Large Language Models

COMP7608B

Lecture Roadmap



Model Lifecycle



Evaluate every checkpoint

Deploy with prompts, tools, retrieval, and monitoring

In-context Learning

ROLE + TASK	You route customer-support tickets. Use the policy below to identify the issue and request only the missing evidence.
POLICY	Billing: payment or refund issues. Account: sign-in issues. Delivery: missing parcels. Two debit entries suggest a possible duplicate, but do not prove two settled payments.
CUSTOMER	Order 472 has one invoice for \$34.90, but my banking app shows two \$34.90 debits. I clicked Pay again after an error. Can you refund one today?
OUTPUT	Return JSON only, with keys team, issue, ask. If settlement is unclear, ask for payment references. Do not promise a refund.

In-context Learning (cont.)

ILLUSTRATIVE RESPONSE

```
{  
  "team": "billing",  
  "issue": "possible_duplicate",  
  "ask": "payment_references"  
}
```

CONTEXT USED

Payment issue: route to billing.

Unclear settlement: retain uncertainty.

Missing references: ask for evidence.

A change in context

Change the routing policy to “all payment issues go to finance.” The expected team becomes finance with the same model weights.

Prompt Components

Prompt component	Support-assistant example	Purpose
Task	Route the ticket and extract evidence	Clarify the decision
Context	Team definitions and known order fields	Supply relevant information
Examples	Billing, delivery, and account cases	Demonstrate the mapping
Output contract	JSON keys: team, issue, ask	Make output usable by software

Few-shot Prompting

Route each ticket. Match the JSON schema in the demonstrations. Ask for evidence only when it is missing.

DEMONSTRATION 1: USER

Order 310 has two settled charges.
Payment references A81 and A82 have
different IDs. Please check the duplicate.

DEMONSTRATION 2: USER

I reset my password but still cannot sign
in. The app displays an error, which I have
not copied.

DEMONSTRATION 1: ASSISTANT

```
{"team": "billing",  
  "issue": "duplicate_charge",  
  "ask": null}
```

DEMONSTRATION 2: ASSISTANT

```
{"team": "account",  
  "issue": "sign_in_failure",  
  "ask": "error_message"}
```

Few-shot Prompting (cont.)

NEW USER MESSAGE

Order 472 has one invoice, but my banking app shows two \$34.90 debits after I retried payment. I have not included the transaction references.

EXPECTED COMPLETION

```
{"team": "billing",  
  "issue": "possible_duplicate",  
  "ask": "payment_references"}
```

PATTERN TRANSFER

Reuse the schema and routing pattern.

Change the issue and evidence request to match this ticket.

Chain-of-Thought Prompting

INSTRUCTION

Solve the new booking problem using a short sequence of calculations. Account for cancellations and reserved seats before allocating new requests.

WORKED DEMONSTRATION IN THE PROMPT

42 seats, 35 bookings, 3 cancellations, 2 reserved seats, 11 new requests.

Active bookings: $35 - 3 = 32$.

Available seats: $42 - 32 - 2 = 8$.

Accept $\min(11, 8) = 8$. Waitlist $11 - 8 = 3$.

NEW PROBLEM

60 seats, 48 bookings, 5 cancellations, and 4 seats reserved. New requests arrive in batches of 6, 5, and 4. Partial batches are allowed. How many can be accepted and waitlisted?

Chain-of-Thought Prompting (cont.)

- 1 Active bookings** $48 - 5 = 43$ occupied seats
- 2 Available seats** $60 - 43 - 4 = 13$ seats for new requests
- 3 Allocation** First 6 fit. Next 5 fit. Only 2 of the last 4 fit.
- 4 Final answer** Accept $6 + 5 + 2 = 13$. Waitlist $4 - 2 = 2$.
- 5 Consistency check** 43 occupied + 4 reserved + 13 new = 60 .
 13 accepted + 2 waitlisted = 15 requests.

Self-consistency

For the same booking problem, sample several complete solutions.

SAMPLED SOLUTIONS

A: cancel, reserve, allocate $\rightarrow$ 13 accepted

B: $60 - (48 - 5) - 4 \rightarrow$ 13 accepted

C: overlook the reserved seats $\rightarrow$ 15 accepted

MAJORITY



13

accepted

In-context Learning and Model Weights

$$y \sim p_{\theta}(y \mid \underbrace{x, \text{instructions, examples}}_1), \quad \underbrace{\theta \text{ fixed}}_2$$

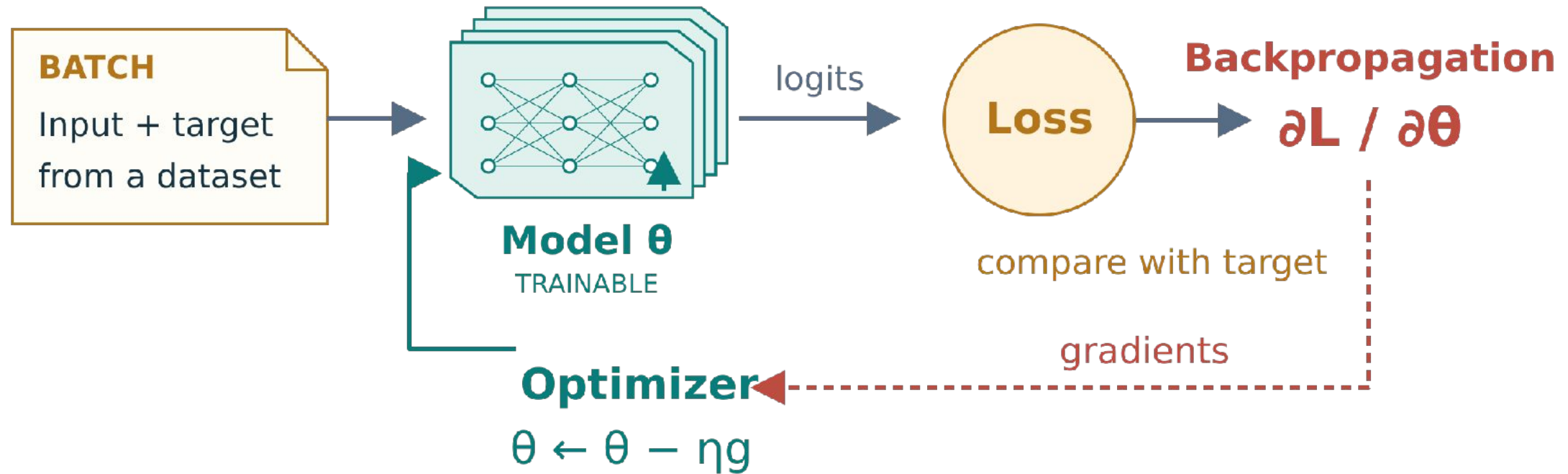
1 Input context

Instructions, examples, and retrieved evidence change the conditional distribution.

2 No optimizer update

Activations and KV cache change. The learned weights stay fixed.

Training Step



Logits and Target Probabilities

TRAINING CONTEXT

The capital of the UK is ... Target token: London

Token	Logit z_i	Softmax p_i	Target q_i
Paris	2.0	0.644	0
London	1.0	0.237	1
Beijing	0.0	0.087	0
Tokyo	-1.0	0.032	0

Cross-entropy Loss

Softmax

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Cross-entropy

$$\mathcal{L} = - \sum_i q_i \log p_i$$

One-hot target

$$\mathcal{L} = -\log p_y = -z_y + \log \sum_j e^{z_j}$$

Logit gradient

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - q_i$$

q assigns 1 to London and 0 to other tokens.

London: 0.237 – 1 = –0.763

Backpropagation

One-weight binary model. Target: class 1.

$$z = wx, \quad p = \frac{1}{1 + e^{-z}}, \quad \mathcal{L} = -\log p \quad (y = 1)$$

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\frac{\partial \mathcal{L}}{\partial z}}_{\text{output error}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{\text{local sensitivity}} = (p - 1)x$$

Input and weight $x = 2, w = 0.5$

Model output $p \approx 0.731, \text{ loss} \approx 0.313$

Weight gradient $(0.731 - 1) \times 2 \approx -0.538$

Gradient Descent

The optimizer uses the gradient of the loss.

$$\underbrace{g = \frac{\partial \mathcal{L}}{\partial w}}_{\text{backpropagation}} \quad \underbrace{w_{\text{new}} = w - \eta g}_{\text{optimizer update}}$$

Same example $w = 0.5, g = -0.538, \eta = 0.1$

$$w_{\text{new}} = 0.5 - 0.1(-0.538) = 0.5538$$

Recompute the prediction with the new weight:

p: 0.731 → 0.752 loss: 0.313 → 0.285

AdamW

Same loss gradient. A different rule for choosing the step.

$$g_k = \nabla_{\theta} \mathcal{L}(\theta_k)$$

Gradient history $\hat{m}$ averages past gradients.

Squared gradients $\hat{v}$ rescales updates for each weight.

$$\theta_{k+1} = \underbrace{(1 - \eta\lambda)\theta_k}_{\text{weight decay}} - \underbrace{\eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}}_{\text{gradient-based step}}$$

η : learning rate λ : decay strength ϵ : numerical stabilizer

Mixed-precision Memory Example

State	Purpose	Bytes / parameter
Model weights	Forward computation	2
Gradients	Direction for the next update	2
FP32 master weights	Accurate parameter updates	4
Adam moments	Gradient history m and v	8

Pre-training

USER REQUEST

Prefix from a technical document:

“To average ten sensor readings, add the readings and divide by”

Random initial weights

“the river function orange...”

After sufficient pre-training

“ten.”

Code and text continuations also become more plausible.

Pre-training Data

Source	Why include it?	If poorly balanced
Web and reference text	Broad topics and language use	Noise, repetition, factual errors
Code	Syntax and executable patterns	Domain bias, duplicated code
Books and long documents	Sustained context and discourse	Source and topic imbalance
Math / educational text	Worked problems and concepts	Narrow problem coverage
Multilingual text	Language coverage	Weak low-resource coverage

Data Preparation



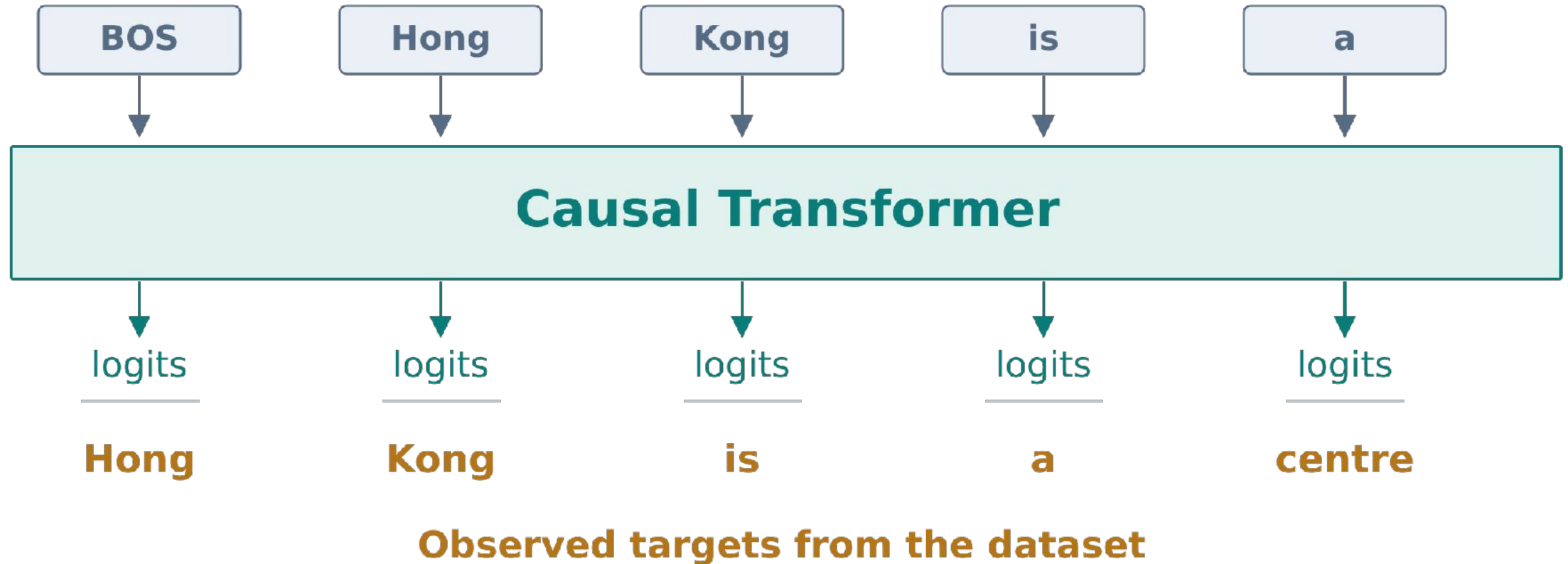
Sampling weights determine how often each source appears.

Keep evaluation items and near-duplicates out of the training corpus.

Next-token Targets

Position	Input token	Target token	Visible prefix
1	BOS	Hong	BOS
2	Hong	Kong	BOS Hong
3	Kong	is	BOS Hong Kong
4	is	a	BOS Hong Kong is
5	a	centre	BOS Hong Kong is a

Teacher Forcing



Pre-training Loss

$$\mathcal{L}_{\text{PT}} = - \underbrace{\frac{1}{M}}_1 \sum_{b,t} \underbrace{m_{b,t}}_2 \underbrace{\log p_{\theta}(x_{b,t+1} \mid x_{b,\leq t})}_3$$

1 Average valid tokens

M = sum of all mask entries.
It is not the number of documents.

2 Choose targets

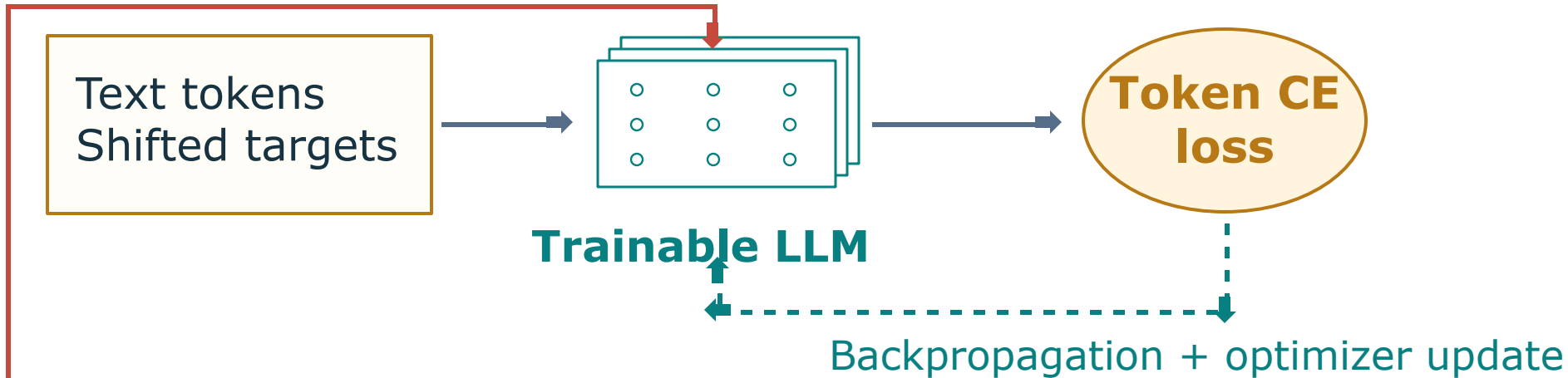
Ignore padding and invalid prediction positions.

3 Next-token likelihood

Penalize low probability on the observed next token.

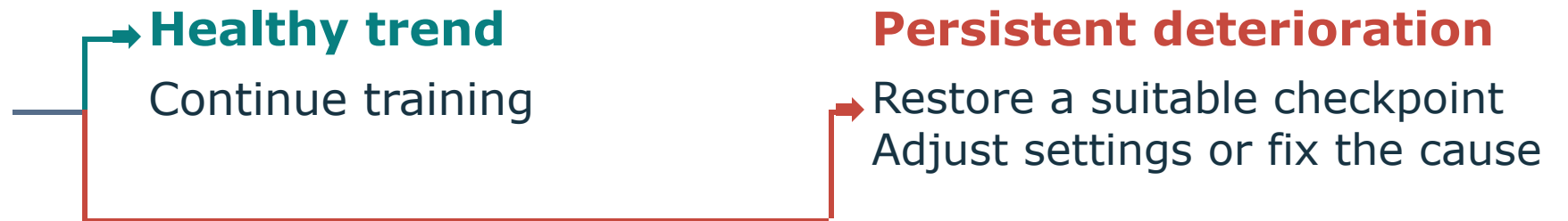
Pre-training Components

Each training step



Periodic checks

Save training state
Check held-out loss



Illustrative Loss Curves

Healthy learning



Both losses fall. Continue while validation improves and compute remains worthwhile.

Overfitting



Validation rises after step 5k. Restore the best checkpoint. Stop, or revise the data and regularization before resuming.

Checkpoint Recovery

Illustrative restart from the last healthy checkpoint



1. PAUSE AND DIAGNOSE

Inspect learning rate, gradient norms, precision, and recent batches after a persistent rise or non-finite loss.

2. RESTORE AND RECHECK

Fix the cause, then restore step 4k. For excessive updates, lower the learning rate. Recheck held-out loss.

Llama 3.1 Training Scale

Quantity	Reported value	Interpretation
Pre-training data	About 15 trillion tokens	Broad text corpus
8B model training	1.46 million H100-80GB GPU-hours	Aggregate device time
Dense compute estimate	$C \approx 6ND \approx 7.2 \times 10^{23}$ FLOPs	$N = 8B, D = 15T$

Training Infrastructure

7B

trainable parameters

× 16 bytes



≈ 112 GB

before activations and buffers

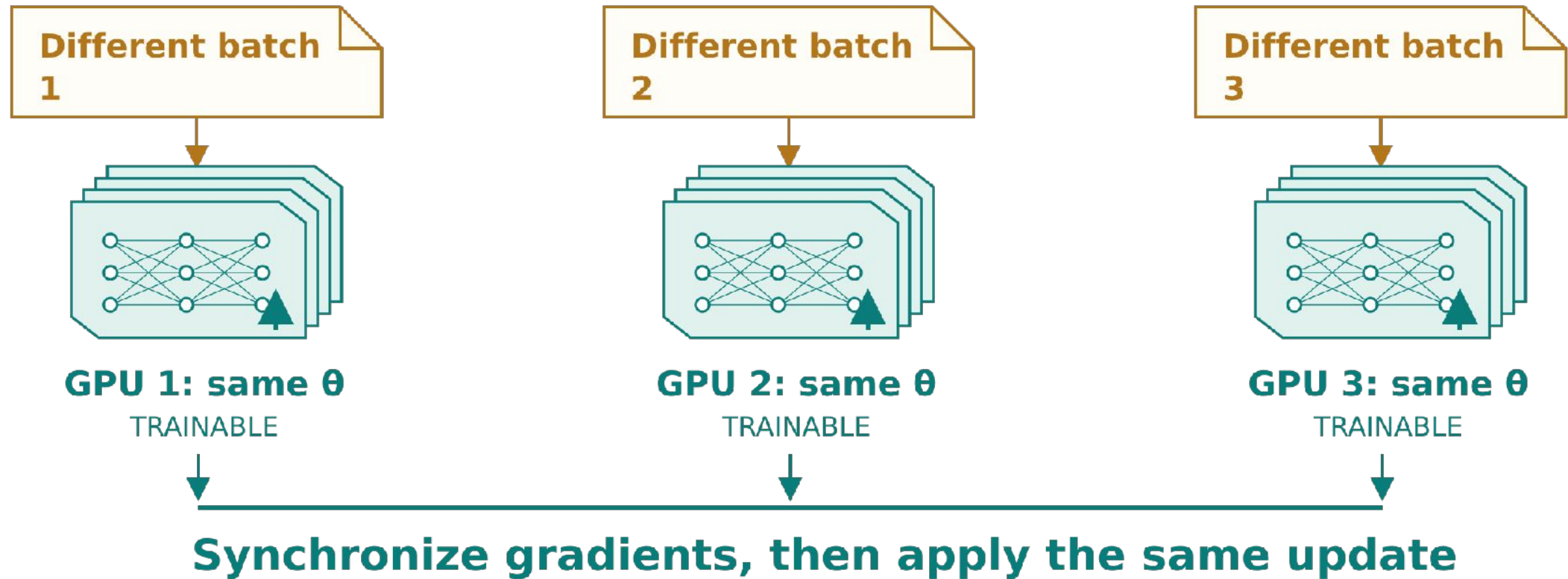
Capacity

Shard training states or split the model.

Throughput

Process more data in parallel, then communicate.

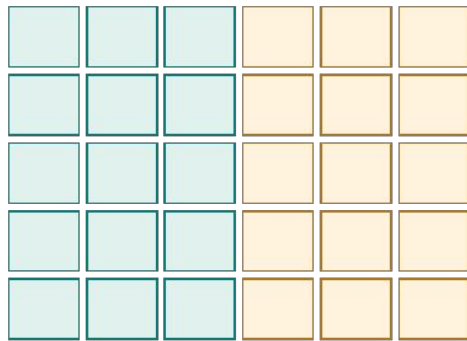
Data Parallelism



Tensor and Pipeline Parallelism

Tensor parallelism

Split one matrix operation

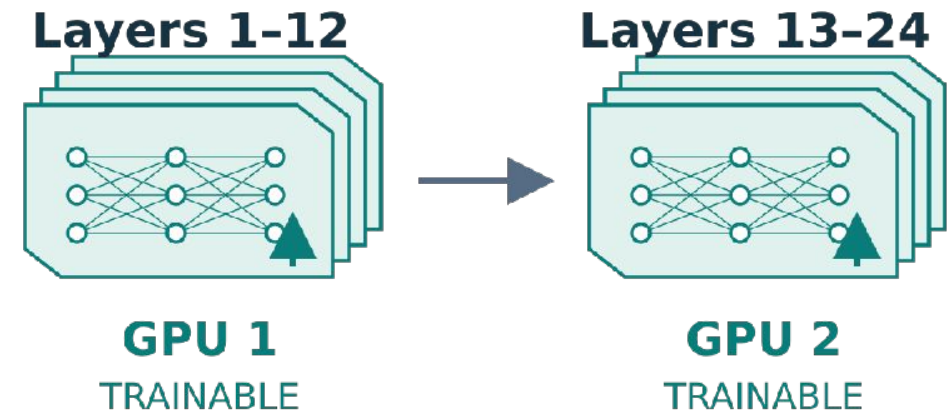


GPU 1 **GPU 2**

Communication within layers

Pipeline parallelism

Split consecutive layer groups



Micro-batches and pipeline bubbles

ZeRO and FSDP

ZeRO: Zero Redundancy Optimizer

FSDP: Fully Sharded Data Parallel

Approach	Partitioned state	Trade-off
Plain data parallel	None: each worker holds replicas	Simple, but memory per GPU stays high
ZeRO stage 1	Optimizer state	Less memory, state synchronization
ZeRO stage 2	Optimizer state + gradients	Further memory savings
ZeRO stage 3 / FSDP	Parameters, gradients, optimizer state	Gather parameters when needed

Training Efficiency

Technique	Characteristic	Main cost or constraint
Activation checkpointing	Recompute selected activations in backward	More compute, lower activation memory
Gradient accumulation	Several micro-batches per optimizer step	Does not accelerate a single GPU by itself
Context parallelism	Distribute sequence positions	Attention communication across devices
Expert parallelism	Place different MoE experts on devices	Token routing and load balancing

Training Frameworks

Framework	Characteristics	Typical use
PyTorch DDP / FSDP	Core distributed and sharding primitives	Custom training code
DeepSpeed	ZeRO, offload, mixed precision, pipeline tools	Fit and train larger models
Megatron-Core / LM	Tensor, pipeline, context, expert parallelism	Large Transformer and MoE training
Accelerate + TRL + PEFT	Launch/backend integration + losses + adapters	Accessible SFT, DPO, and RL experiments

Mid-training

Change	Data or technique	Reason
Higher-quality mixture	Educational text and worked examples	Improve useful signal per token
Domain / capability focus	Code, math, domain documents	Develop targeted coverage
Longer context	Long documents and position handling	Use information across more tokens
Retention	General-data replay and held-out checks	Detect capability regressions

Domain Adaptation

USER REQUEST

Training text: "A lumet is a sensor reading averaged over ten seconds."

Test query, manual withheld: "What is a lumet?"

Before domain exposure

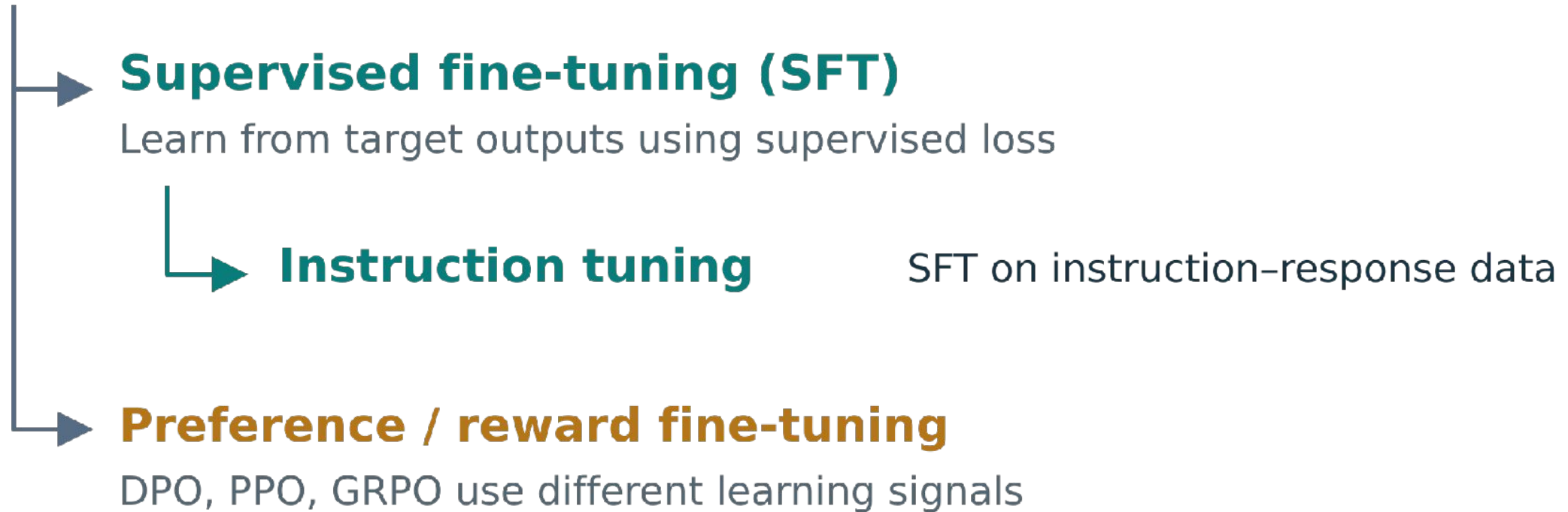
"I do not recognize this term."

Desired answer after adaptation

"A lumet is the sensor reading averaged over ten seconds."

Fine-tuning and Instruction Tuning

Fine-tuning Adapt an existing checkpoint by updating parameters



Instruction Tuning

USER INSTRUCTION

Convert the customer message into JSON for a support agent.

1. Extract every issue with an exact supporting quote.
2. Use categories safety, billing, delivery, or account.
3. Set priority to high only when there is a safety issue.
4. Include order_id and next_actions. Use null for missing IDs.
5. Return JSON only. Do not promise a refund or replacement.

CUSTOMER MESSAGE

Order 472 arrived yesterday. The charger sparked twice, so I stopped using it. My bank shows two \$34.90 debits, but I received one invoice. Can you refund one payment and replace the charger today?

Instruction Tuning (cont.)

CHECKED ASSISTANT TARGET

```
{  
  "issues": [  
    {"category": "safety",  
     "evidence": "The charger sparked twice"},  
    {"category": "billing",  
     "evidence": "two $34.90 debits"}  
  ],  
  "priority": "high",  
  "order_id": 472,  
  "next_actions": ["keep_charger_out_of_use",  
                   "request_payment_references",  
                   "review_replacement_request"]  
}
```

What is learned?

Follow all constraints.

Ground fields in the message.

Produce valid structured output.

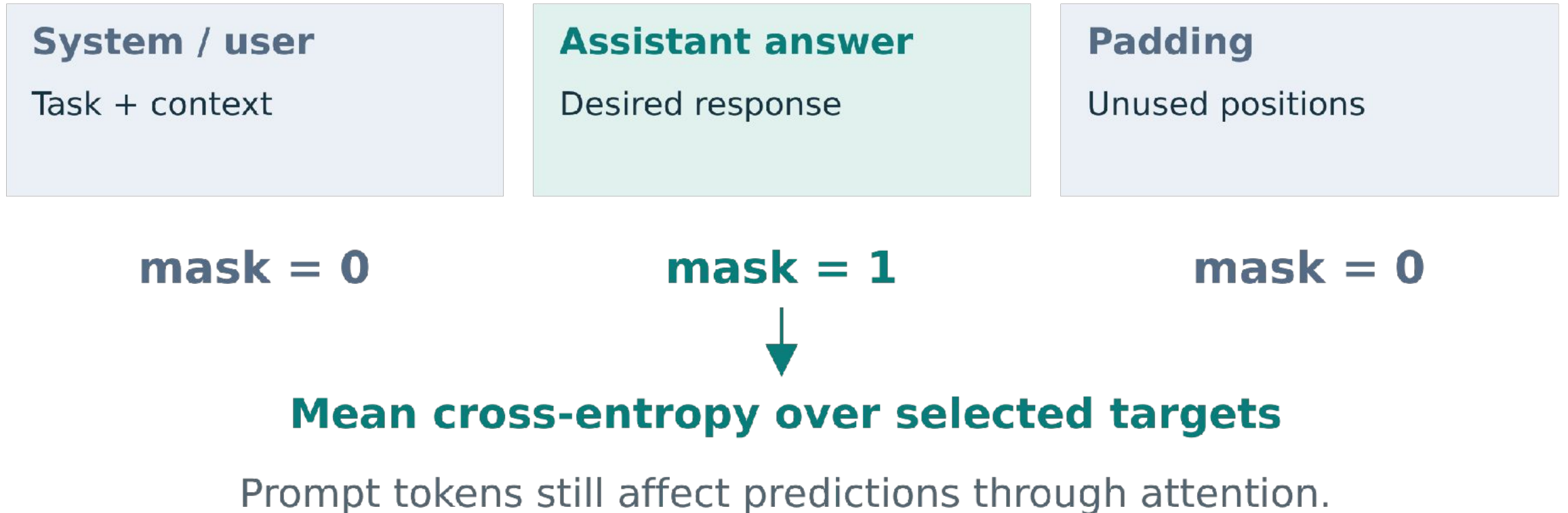
SFT Data Coverage

Data component	What it teaches	What to inspect
Task instructions	Summarize, classify, transform	Correctness and diversity
Multi-turn conversations	Use history and follow roles	Role boundaries and context
Code / worked solutions	Solve and explain tasks	Executable or checked answers
Preference / safety behavior	Tone, refusals, uncertainty	Over-refusal and inconsistencies
Languages and formats	Serve the intended users	Balanced coverage and schemas

SFT Data Sources

Source of answers	Open example	What to inspect
Human authors	Dolly 15K; OpenAssistant	Task coverage and annotation quality
A teacher model	Stanford Alpaca	Teacher errors and repeated templates
Human + synthetic mixture	Tülu 3 SFT mixture	Coverage, source tags, filtering
Your task experts	Written or corrected application examples	Realistic inputs and accepted outputs

Response Masking



SFT Loss

$$\mathcal{L}_{\text{SFT}} = - \underbrace{\frac{1}{\sum_t m_t}}_3 \sum_t \underbrace{m_t}_1 \underbrace{\log \pi_{\theta}(y_t \mid x, y_{<t})}_2$$

1 Response mask

Select assistant targets. The prompt remains visible as context.

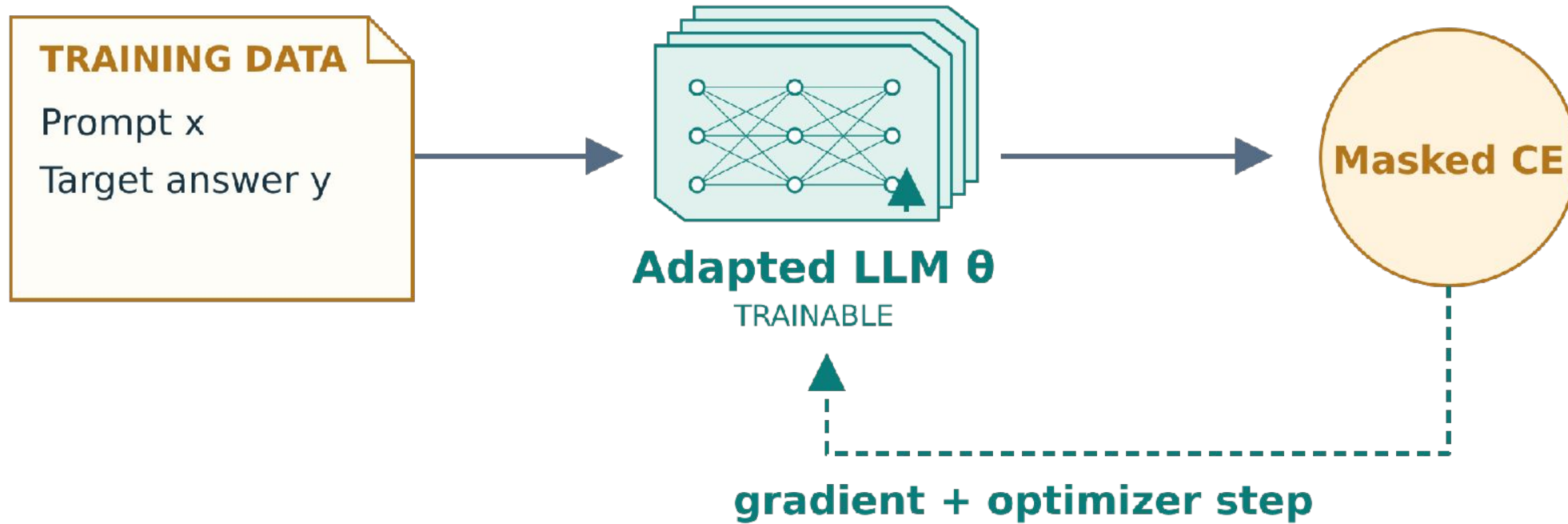
2 Demonstration target

Predict the next token of the provided answer, given its observed prefix.

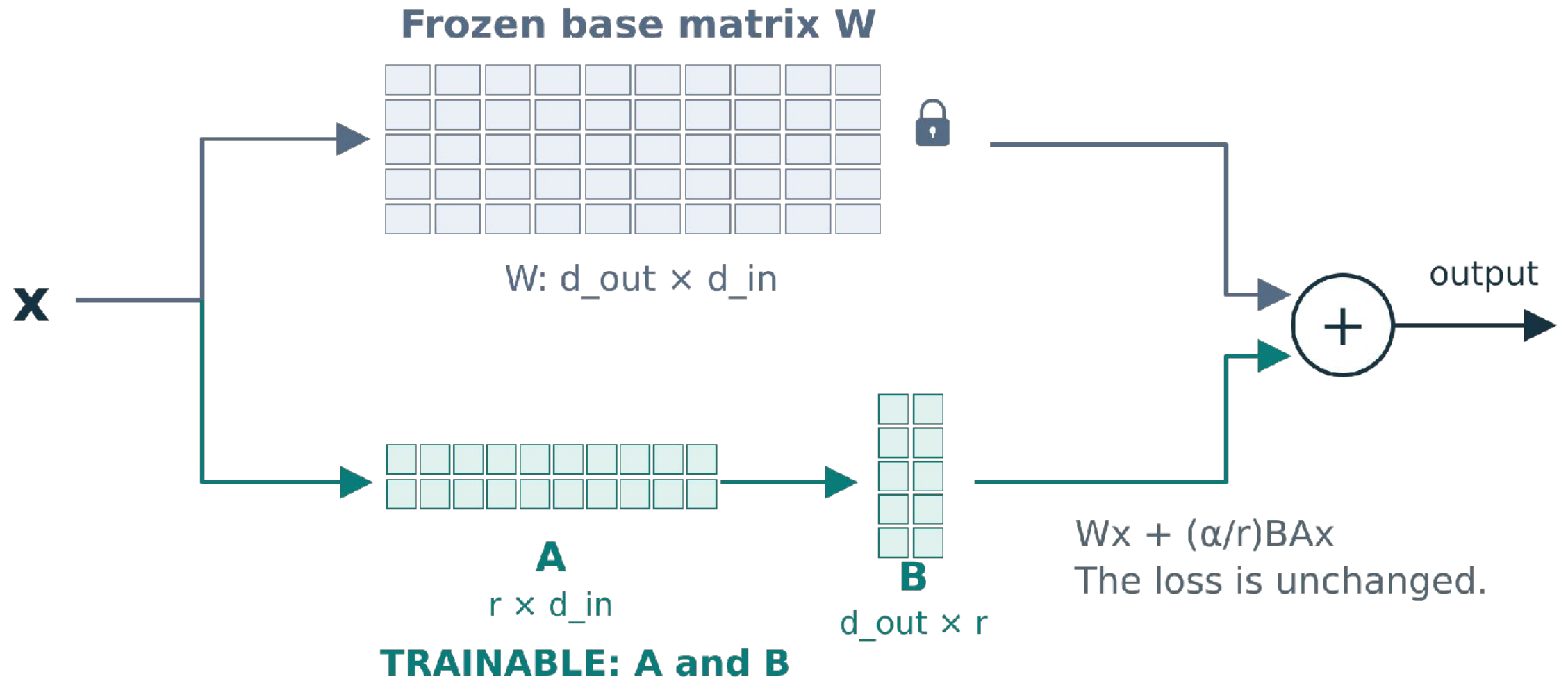
3 Normalization

Average only the selected tokens. Different weighting changes the objective.

SFT Components



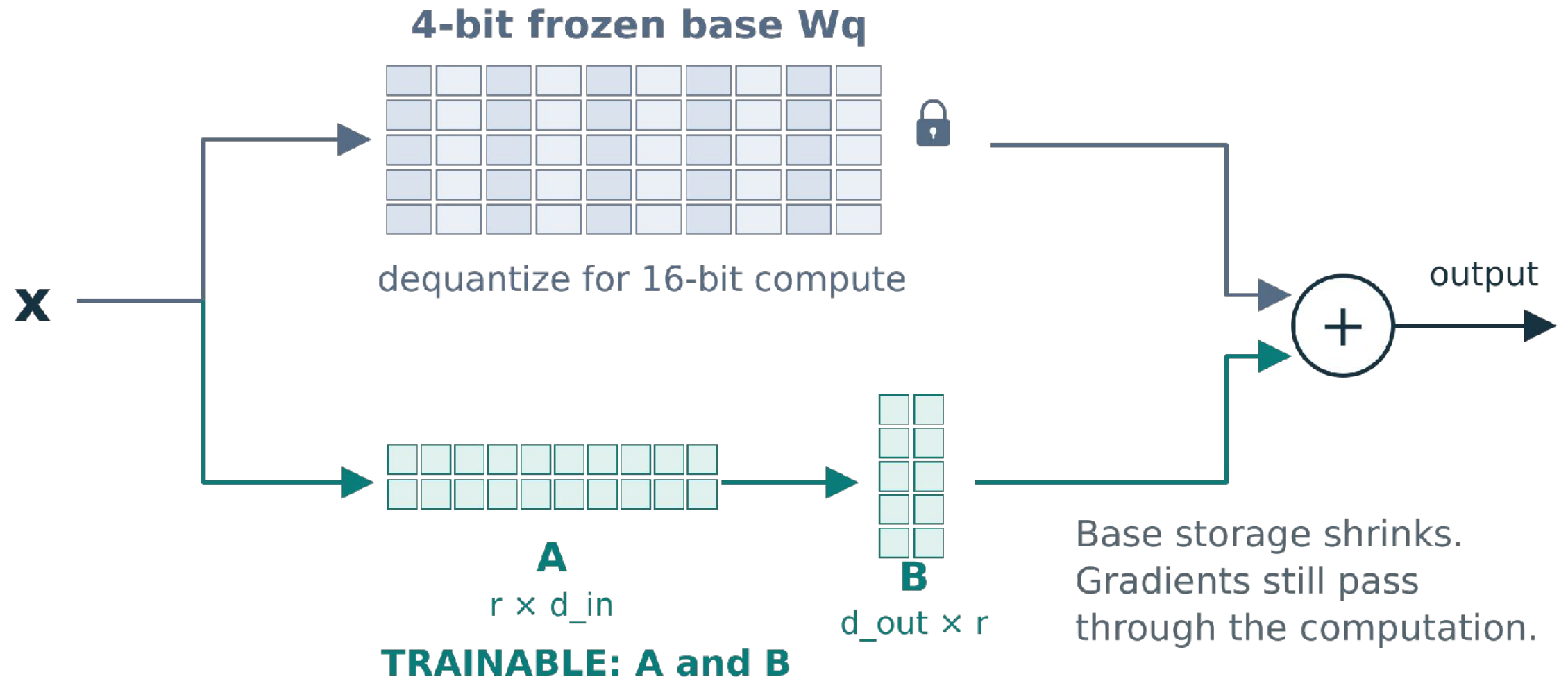
Low-Rank Adaptation (LoRA)



LoRA Parameter Count

Parameter block	Count	Updated?
Original W	$4,096 \times 4,096 = 16,777,216$	Frozen
Adapter A	$8 \times 4,096 = 32,768$	Trainable
Adapter B	$4,096 \times 8 = 32,768$	Trainable
Total adapter	$65,536 \approx 0.39\%$ of W	Small optimizer state

Quantized LoRA (QLoRA)



Objectives and Trainable Parameters

Strategy	Trainable state	Typical trade-off
Full tuning	All selected model weights	Most flexibility; high memory
Partial tuning	Selected layers / blocks	Smaller update space
LoRA	Low-rank adapter matrices	Small checkpoints; rank choice
QLoRA	Adapters on a quantized base	Lower base memory; extra machinery

SFT Evaluation

USER REQUEST

Held-out ticket: "Order 901 was charged twice, but the replacement charger works safely."

Possible failure before tuning

"Urgent safety issue.
Stop using the charger."

Desired response after SFT

```
{"issues": ["billing"],  
"urgency": "normal",  
"next_step": "check_payment"}
```

SFT Training Cost

Experiment	Data and setup	What it establishes
Stanford Alpaca	52K demonstrations; LLaMA 7B 8 × A100-80GB for about 3 h	About 24 GPU-hours for that reported tuning run
QLoRA	65B model; 4-bit frozen base One 48 GB GPU	Memory feasibility under the paper's configuration
Your experiment	Examples × tokens × epochs Full tuning or adapters	Measure peak memory, wall time, and validation

SFT Trade-offs

Strength

Direct, inspectable demonstrations.
Efficient behavior and format adaptation.

Failure modes

Bad targets, overfitting, and forgetting.
Fluent imitation can conceal incorrect reasoning.

Cost control

Improve coverage and labels first.
Track trainable state, tokens, and regressions.

Reinforcement Learning from Human Feedback

RLHF

Train the assistant to produce answers people prefer.

Problem	Many plausible responses differ in usefulness, accuracy, and tone.
Human feedback	Compare two answers to the same prompt.
Learning signal	A learned scoring rule guides later practice.

Preference Example

PROMPT

I trained a support-ticket classifier. It gets 99% accuracy on the training set but 68% on tickets from last month. About 92% of last month's tickets are routine and 8% are urgent.

My manager wants to launch because the training accuracy is 99%. Explain whether that evidence is sufficient and recommend two checks before deployment. Distinguish what the numbers show from what still needs investigation.

PREFERENCE RUBRIC

Prefer correct use of the evidence, appropriate uncertainty, and actionable evaluation advice. Judge the whole response, not its length or confidence.

Preference Example (cont.)

REJECTED RESPONSE

The training score shows the model has learned the task, and the gap proves overfitting. First, add dropout and train on more examples. Second, keep tuning against last month's tickets until accuracy reaches 95%. Once that threshold is reached, launch the model: the improved score will confirm that it generalizes.

CHOSEN RESPONSE

Not yet. Always predicting routine would score 92%, above this model's 68%. First, audit the split, labels, and whether the recent tickets match deployment. Second, measure urgent-ticket precision and recall on a separate, untouched test set. The gap alone does not prove overfitting.

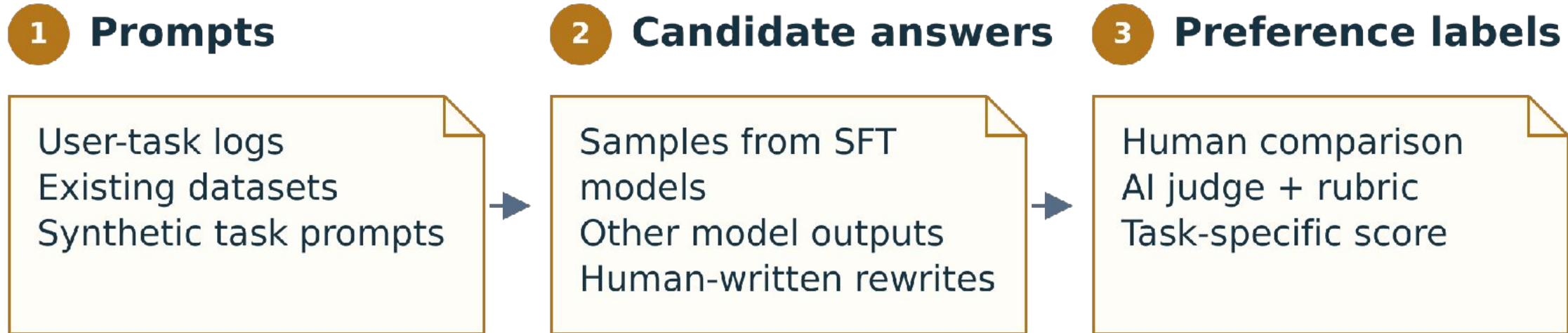
Basis for the preference

The chosen response uses the baseline, avoids an unsupported diagnosis, and protects an independent evaluation set.

RLHF Datasets

Record	How it is obtained	What it trains
Demonstration: (x, y^*)	Human authors or checked teacher answers	SFT creates the starting assistant
Comparison: (x, y^+, y^-)	Candidate answers ranked by humans	Fits a reward model, or trains DPO
Rollout prompt: x	Representative user tasks or authored prompts	Policy generates fresh answers for PPO

Preference Data Collection



Result: (prompt x , chosen answer y^+ , rejected answer y^-)

Human feedback: HH-RLHF AI feedback: UltraFeedback

Preference Data Quality

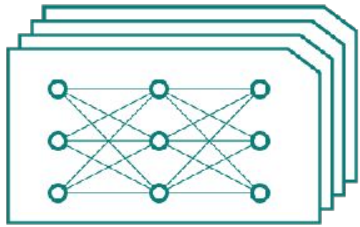
Component	Why include it?	Failure to avoid
Prompt distribution	Reflect intended users and tasks	Only easy or narrow prompts
Chosen / rejected responses	Expose a meaningful quality gap	Wrong or ambiguous labels
Rubric and annotators	Define correctness, tone, and safety	Hidden disagreement or style bias
Hard and diverse pairs	Distinguish strong alternatives	Length as a shortcut for quality

Preference and Reward Methods

Method	Feedback	Learning idea
PPO Proximal Policy Optimization	Fresh answers scored by a reward model	Practice, compare with expectations, then update cautiously
DPO Direct Preference Optimization	Stored chosen / rejected answer pairs	Learn directly from comparative corrections
GRPO Group Relative Policy Optimization	Scores for several answers to one prompt	Compare answers within the group, then update cautiously

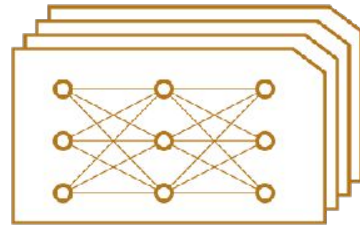
Proximal Policy Optimization (PPO)

The policy writes, a judge scores, a critic predicts, and a reference anchors. PPO favors better-than-expected choices with cautious updates.



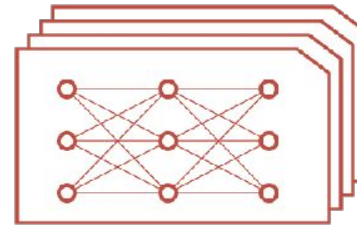
Policy

Writes the answer



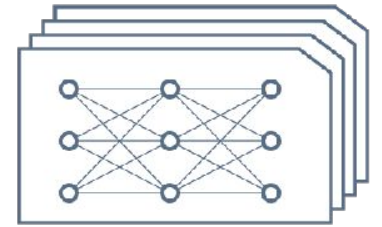
Reward

Grades the
answer



Value / critic

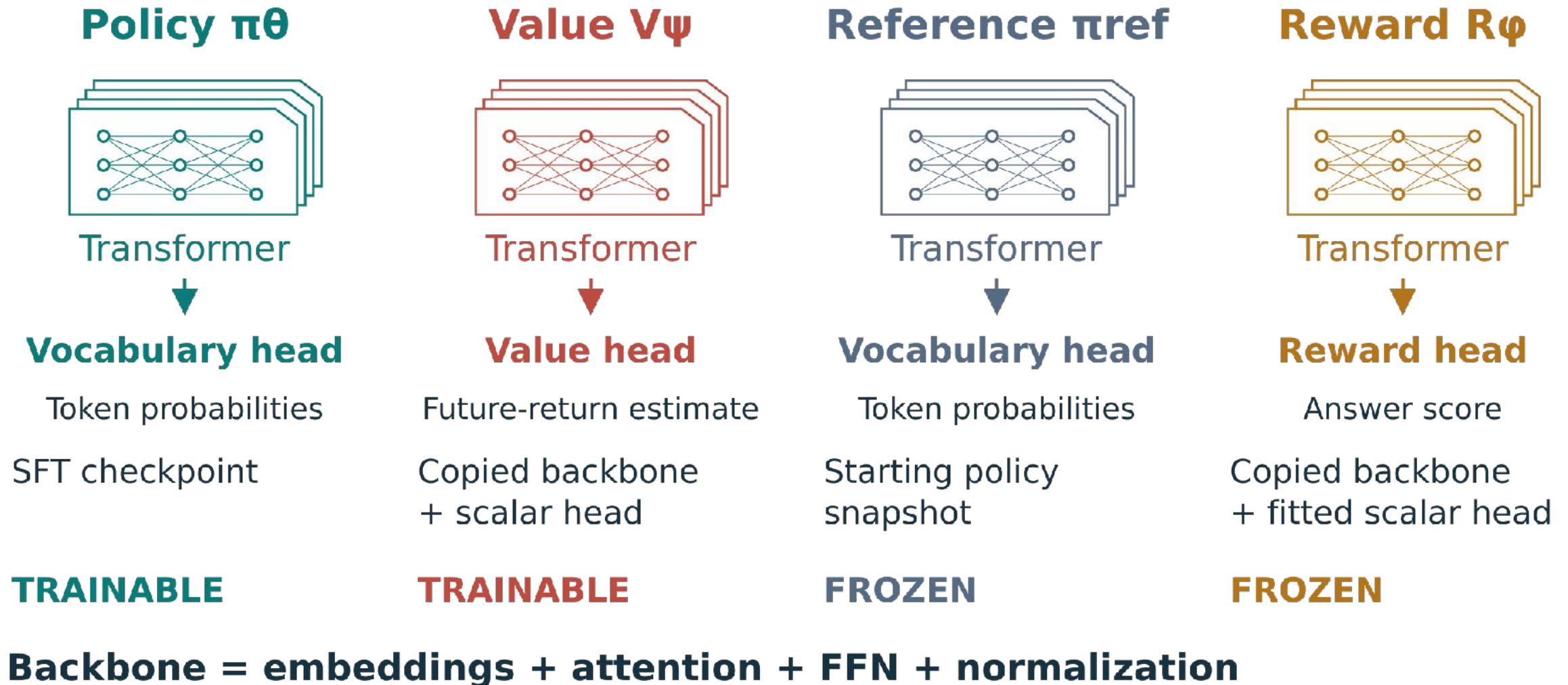
Predicts future
reward



Reference

Anchors behavior

PPO Components



PPO Training Stages

1 Train the judge



Reward-model loss

Updates the reward model

2 Train the assistant

1 Generate

Policy samples
answers

2 Score

Frozen judge +
reference

3 Update

Policy and critic learn

Reward Model

PROMPT x

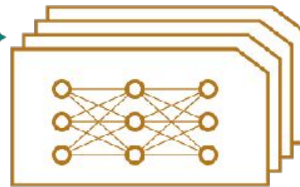
My classifier gets 99% training accuracy and 68% on new data. Is it ready to use?

CHOSEN ANSWER y^+

"Check a representative held-out set."

REJECTED ANSWER y^-

"99% is excellent. The model is ready."



One reward model, used twice

Training should give the chosen answer a higher scalar score.

Bradley–Terry Preference Model

Convert a score difference into a preference probability.

$$P(y^+ \succ y^- \mid x) = \sigma(R_\phi(x, y^+) - R_\phi(x, y^-))$$

$$\sigma(u) = \frac{1}{1 + e^{-u}}$$

Score gap -1

P ≈ 0.269

Score gap 0

P = 0.500

Score gap +1

P ≈ 0.731

P is the probability that the chosen answer wins.

Reward-model Loss

Binary cross-entropy on a human comparison

$$\mathcal{L}_{\text{pair}} = -[c \log P + (1 - c) \log(1 - P)]$$

For an ordered chosen / rejected pair, $c = 1$.

$$\mathcal{L}_{\text{RM}} = -\log P = -\log \sigma(R_{\phi}(x, y^{+}) - R_{\phi}(x, y^{-}))$$

Unit

One loss per response pair, then a batch mean.

Update

Reward weights ϕ , before the PPO phase.

Policy Rollout

PROMPT x

My classifier gets 99% training accuracy and 68% on new data. Is it ready to use?



State s_t

Prompt + tokens already generated

Action a_t

The next sampled token

Rollout

Complete sampled response. EOS marks its end.

Policy Snapshots

Reference π_{ref} The starting SFT model. Fixed throughout PPO.

Old policy π_{old} The policy that collected this rollout batch.

Current policy π_{θ} The policy being updated on this batch.

Beginning of a round

$$\pi_{\theta} = \pi_{\text{old}}$$



After optimizer steps

$$\pi_{\theta} \neq \pi_{\text{old}}$$

Save old action probabilities. Refresh them with the next rollout batch.

Reference Model

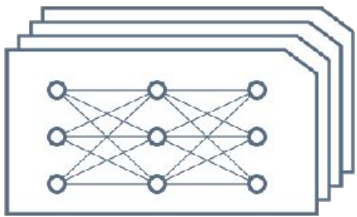
SCORER FAILURE

Long answers receive high scores, even when a short answer is better.



Reward alone

The policy may become verbose everywhere.



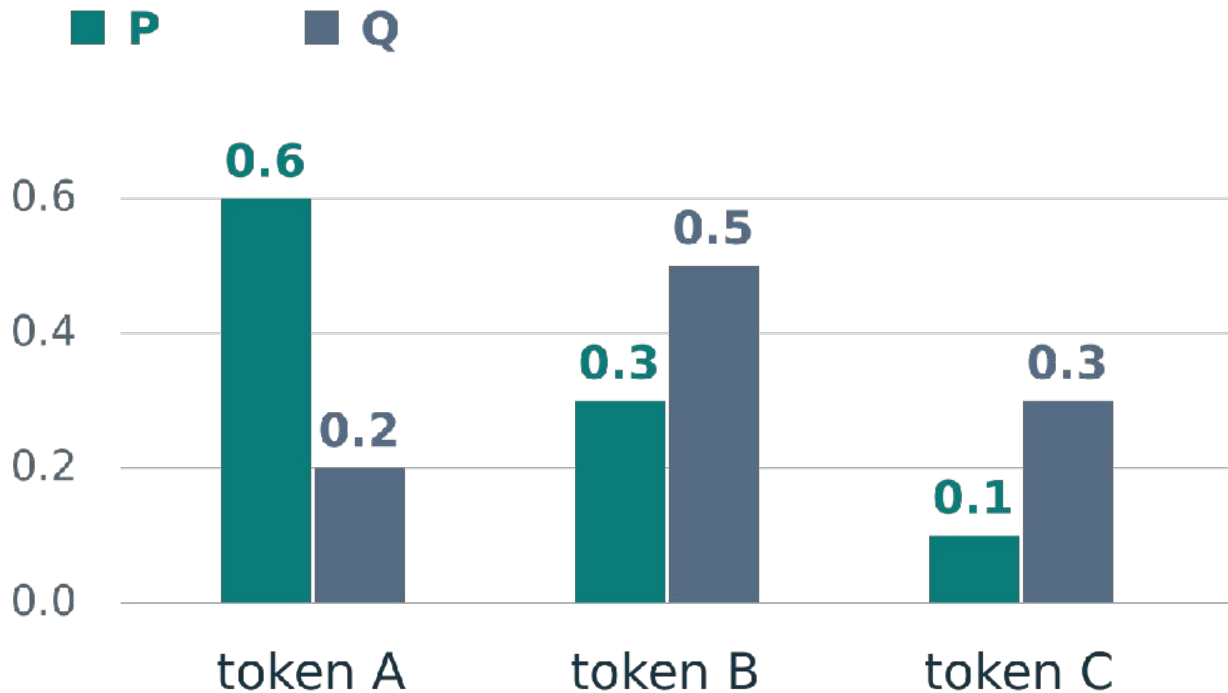
Reference

Compare next-token distributions with the starting assistant.

Make large departures costly while allowing useful improvements.

Kullback–Leibler (KL) Divergence

Probability of the next token



$$D_{\text{KL}}(P||Q) = \sum_i P_i \log \frac{P_i}{Q_i}$$

P = Q: KL = 0

Here: KL $\approx$ 0.396 nats

Distribution mismatch

Direction matters: P || Q differs from Q || P.

Reference Penalty

Score the same sampled token under old and reference policies.

$$k_t = \log \frac{\pi_{\text{old}}(a_t \mid s_t)}{\pi_{\text{ref}}(a_t \mid s_t)} \quad r_t^{\text{KL}} = -\beta k_t$$

Sampled log-ratio k_t How much more or less likely this token became.

Strength β How strongly drift affects the reward.

Example: old probability 0.6, reference probability 0.2, $\beta = 0.1$

$$k_t = \log 3 \approx 1.10 \quad \Rightarrow \quad r_t^{\text{KL}} \approx -0.11$$

Shaped Rewards

$$r_t = \underbrace{\mathbf{1}[t = T]R_\phi(x, y)}_{\text{answer score at the last token}} - \underbrace{\beta k_t}_{\text{reference penalty at every token}}$$

- R** One score for the completed answer
- r_t** Reward assigned at token position t
- $\mathbf{1}[t = T]$** 1 at the last token, 0 at other positions

Example: $R = 1$, $\beta = 0.1$, $k = [0.2, 0.3, 0.5, 0]$

Token rewards $r = [-0.02, -0.03, -0.05, 1.00]$

Returns

Return = all future rewards from this position onward.

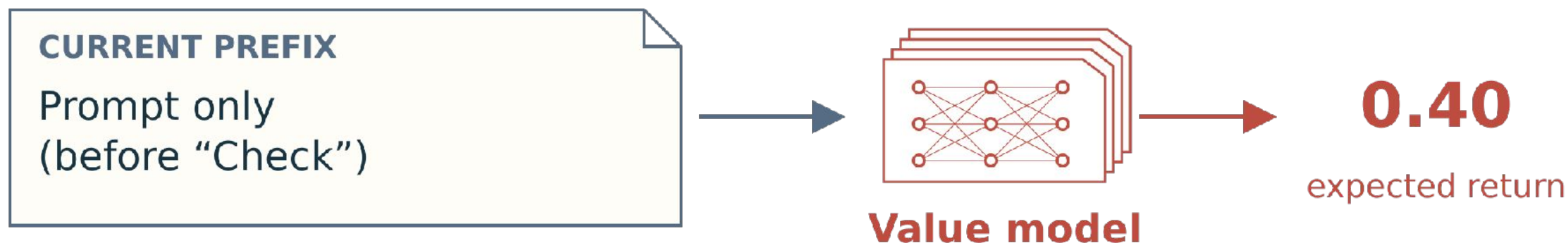
$$G_t = \sum_{u=t}^T r_u$$

Token	Check	unseen	data	EOS
Immediate r_t	-0.02	-0.03	-0.05	1.00
Return G_t	0.90	0.92	0.95	1.00

$$G_1 = -0.02 - 0.03 - 0.05 + 1.00 = 0.90$$

Value Model

The critic predicts the reward still to come from a partial answer.



Reward model Grades the completed answer.

Value model Predicts future return from each prefix.

A return estimate is a real number, not necessarily a probability.

Value Loss

Fit the prediction to the observed return.

$$\mathcal{L}_{\text{value}}(\psi) = \frac{1}{T} \sum_{t=1}^T \left(\underbrace{V_{\psi}(s_t)}_{\text{current prediction}} - \underbrace{G_t}_{\text{stored target}} \right)^2$$

Example

Prediction 0.40, observed return 0.90

$$(0.40 - 0.90)^2 = 0.25$$

Update the critic weights ψ to reduce this prediction error.

Advantage

How much better or worse was this outcome than expected?

$$A_t = \underbrace{G_t}_{\text{observed return}} - \underbrace{v_t}_{\text{saved critic prediction}}$$

**Better than
expected**

$$0.90 - 0.40 = +0.50$$

**Worse than
expected**

$$0.30 - 0.40 = -0.10$$

Keep A_t fixed during policy updates.

Policy-gradient Loss

Cross-entropy, weighted by the sampled outcome

$$\mathcal{L}_{\text{PG}}(\theta) = -\frac{1}{T} \sum_t \underbrace{A_t}_{\text{fixed weight}} \underbrace{\log \pi_{\theta}(a_t | s_t)}_{\text{trainable token score}}$$

$A_t > 0$

Increase support for the sampled token.

$A_t < 0$

Decrease support for the sampled token.

$$\frac{\partial \ell_t}{\partial z_i} = A_t(p_i - q_i)$$

q marks the sampled token. The $p - q$ error is scaled by A_t .

PPO Probability Ratio

Use stored answers for several optimizer steps.

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Old: 0.20
Current: 0.22
Ratio: 1.10

$$\mathcal{L}_{\text{PG}} = -\frac{1}{T} \sum_t A_t \log \pi_{\theta}(a_t \mid s_t)$$

$$\mathcal{L}_{\text{ratio}} = -\frac{1}{T} \sum_t \rho_t(\theta) A_t$$

At $\pi_{\theta} = \pi_{\text{old}}$, both give the same policy gradient.

PPO Clipping

Repeated updates can overreact to the same sampled answers.

$$\mathcal{L}_{\text{policy}} = -\frac{1}{T} \sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)$$

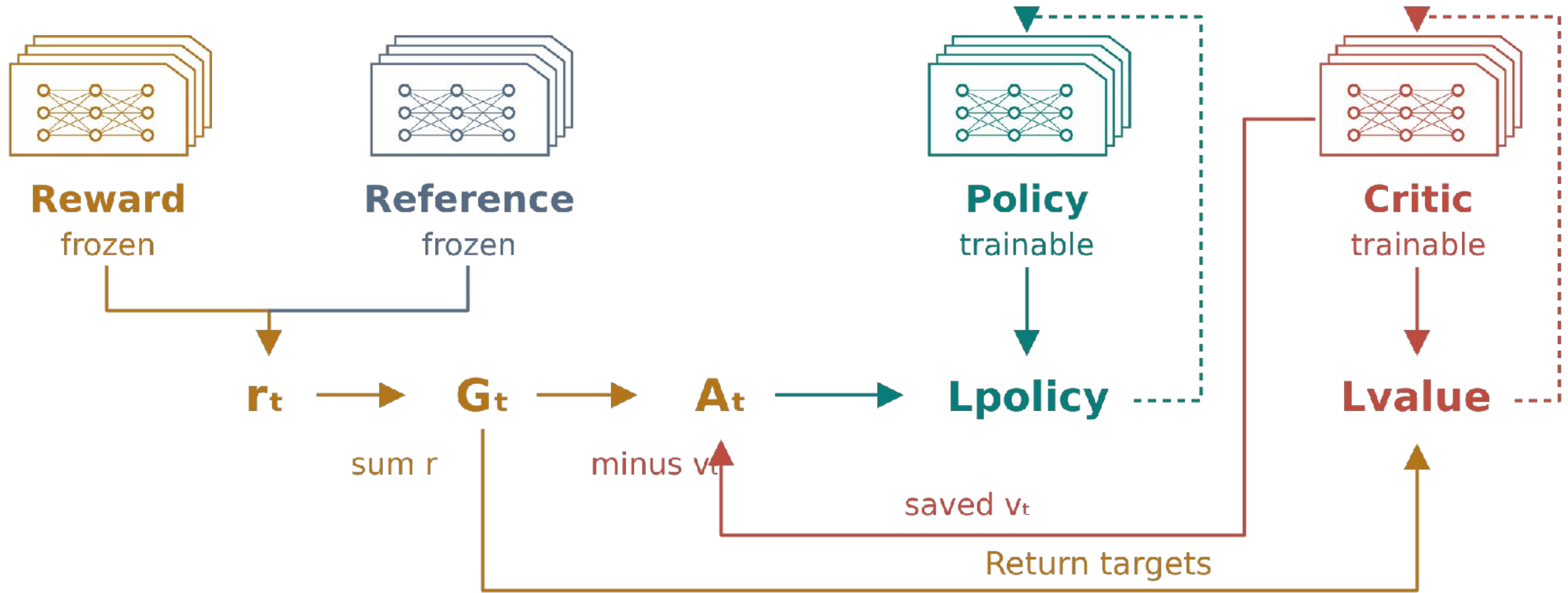
Unclipped term $\rho_t A_t$ keeps rewarding favorable changes.

Clipped term Stops extra incentive beyond the chosen threshold.

$\epsilon = 0.2$ Compare against ratios 0.8 and 1.2.

$\mathcal{L}_{\text{policy}}$ replaces the earlier policy surrogate.

PPO Loss Flow



Solid arrows: stored signals and predictions

Dashed arrows: optimizer updates

PPO Loss

$$r_t = \mathbf{1}[t = T]R_\phi - \beta k_t \quad G_t = \sum_{u=t}^T r_u \quad A_t = G_t - v_t$$



$$\mathcal{L}_{\text{PPO}}(\theta, \psi) = \underbrace{\mathcal{L}_{\text{policy}}(\theta; A)}_{\text{better token choices}} + \underbrace{c_V \mathcal{L}_{\text{value}}(\psi; G)}_{\text{better return predictions}}$$

Policy θ

Uses fixed advantages A and current token scores.

Value ψ

Fits current values to fixed return targets G .

c_V weights the critic loss. Rewards enter through G and A .

PPO Update Cycle

1 Collect

Generate answers. Save old token probabilities.



2 Prepare

Compute rewards, returns, and advantages.



3 Optimize

Update policy and critic on this batch.



Next round: collect fresh answers with the updated policy.

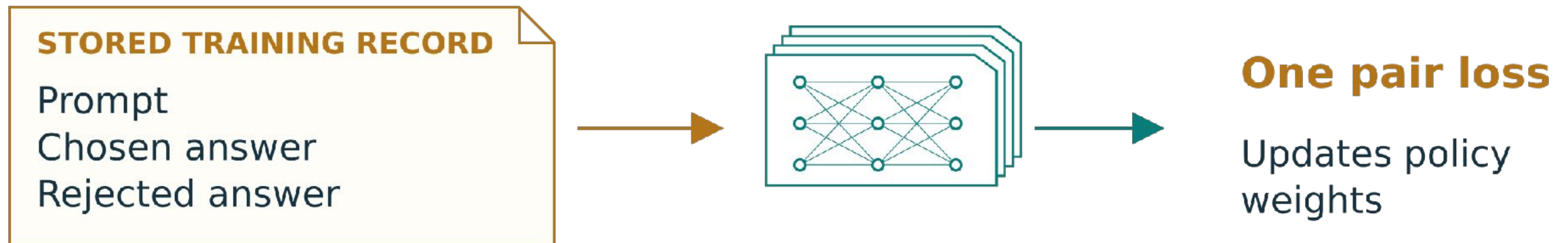
InstructGPT Training Data

Dataset	Training prompts	Used for
SFT	12,725	Demonstration fitting
Reward modeling	33,207	Human-ranked response comparisons
PPO	31,144	Fresh policy rollouts during RL

Direct Preference Optimization (DPO)

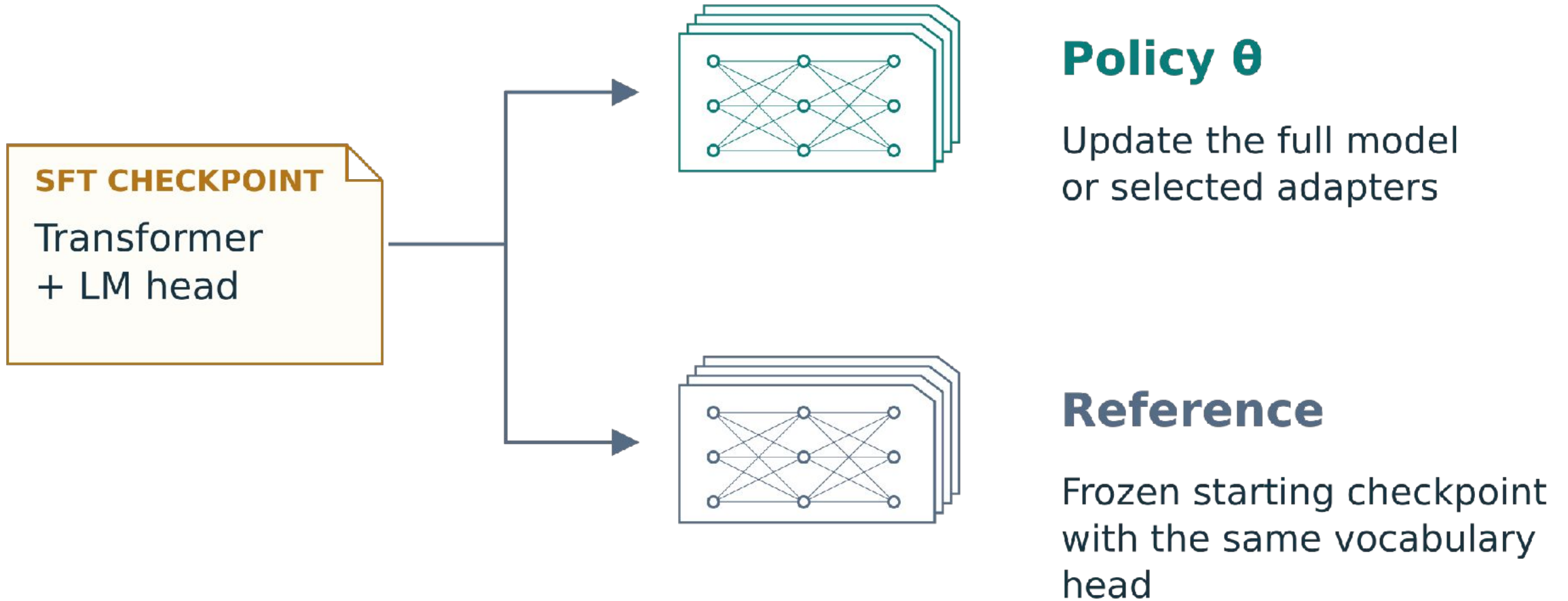
Learn directly from comparative corrections:

“Prefer this answer to that one, more strongly than the starting model did.”



No separate reward fitting or critic training in standard offline DPO.

DPO Components



DPO Sequence Scores

Both models score the same stored response tokens.

$$\ell_{\theta}(y) = \log \pi_{\theta}(y \mid x) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t})$$

Chosen y^+

token log p + token log p + ...

→ $\ell_{\theta}(y^+)$

Rejected y^-

token log p + token log p + ...

→ $\ell_{\theta}(y^-)$

Repeat under the frozen reference to get four response scores.

DPO and Reward Modeling

Same Bradley-Terry comparison model

$$P(y^+ \succ y^- \mid x) = \sigma(\text{score}(y^+) - \text{score}(y^-))$$

PPO route: a separately fitted reward model supplies the scores.

DPO route: policy / reference probabilities supply an implicit score.

$$\hat{R}_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

DPO Loss

$$\Delta_{\theta} = \ell_{\theta}(y^{+}) - \ell_{\theta}(y^{-}) \quad \Delta_{\text{ref}} = \ell_{\text{ref}}(y^{+}) - \ell_{\text{ref}}(y^{-})$$

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left[\beta \left(\underbrace{\Delta_{\theta}}_{\text{policy margin}} - \underbrace{\Delta_{\text{ref}}}_{\text{reference margin}} \right) \right]$$

β

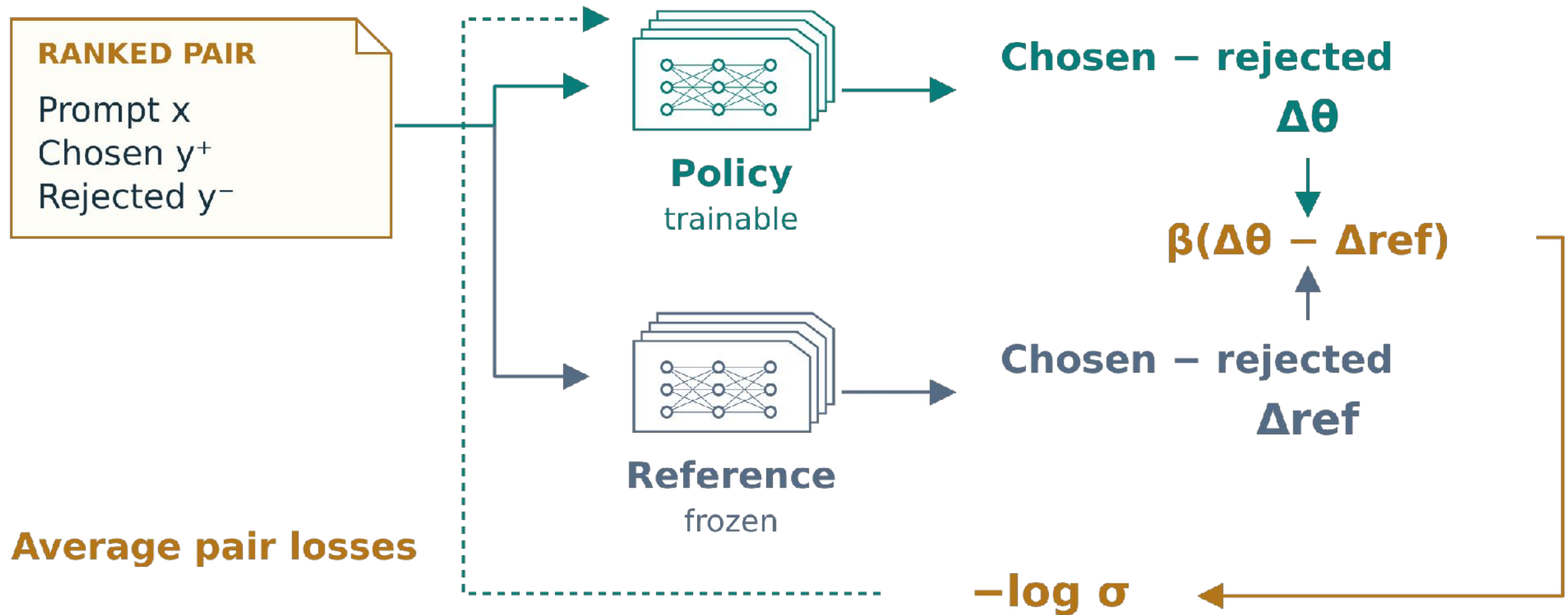
Scale inherited from the reward-KL formulation.

$-\log \sigma$

Cross-entropy for “the chosen answer wins.”

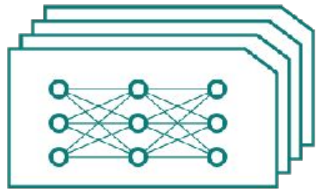
One loss per pair. Average the pairs and update only θ .

DPO Loss Flow

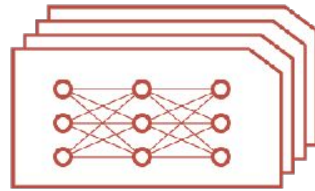


PPO and DPO

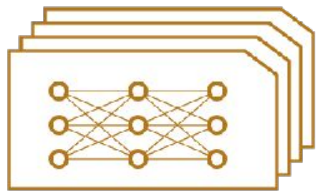
PPO with human-feedback reward



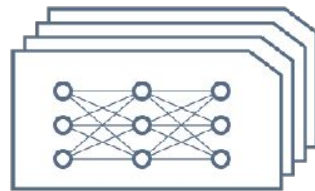
Policy
train



Critic
train



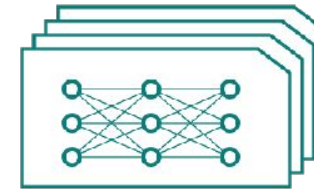
Reward
frozen



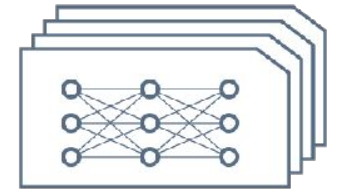
Reference
frozen

Fresh answers + two trainable roles

Standard offline DPO



Policy
train



Reference
frozen

Stored comparisons

One pair loss

No critic

No rollout generation per update

Preference Learning Trade-offs

- Reward hacking** The policy learns shortcuts that score well, such as verbosity or flattering the user.
- Distribution shift** The policy produces responses outside the reward model's reliable training distribution.
- What to measure** Held-out human preference, factuality, and safety, along with KL, length, and general regressions.

Other Preference Methods

Method	Training signal	Distinctive idea
ORPO Odds Ratio Preference Optimization	Chosen / rejected pairs	SFT plus an odds-ratio preference term
KTO Kahneman–Tversky Optimization	Desirable / undesirable labels	Learn from binary feedback without paired alternatives
SimPO Simple Preference Optimization	Chosen / rejected pairs	Length-normalized implicit reward and a target margin

Reasoning Training

Target: solve new problems reliably, including multiple steps.

Prompting	Show a useful approach in the context. Weights stay fixed.
Reasoning SFT	Learn from checked worked solutions.
Reasoning RL	Try solutions and learn from scored outcomes.

Decimal Comparison

USER REQUEST

“Which number is larger: 0.11 or 0.9? Explain briefly.”

Illustrative failure

“0.11 is larger because 11 is greater than 9.”

What went wrong?

The answer compares digit strings as integers and ignores decimal place value.

Decimal Comparison (cont.)

Prompt: Compare 0.11 and 0.9. Show a common place-value representation, calculate the difference, and check the result.

- 1 Align the precision** $0.11 = 11/100.$ $0.9 = 0.90 = 90/100.$
- 2 Compare equal units** 90 hundredths $>$ 11 hundredths, so $0.9 > 0.11.$
- 3 Calculate the gap** $0.90 - 0.11 = 0.79.$
- 4 Check the conclusion** $0.11 + 0.79 = 0.90.$
Final: 0.9 is larger by 0.79.

Reasoning Transfer

NEW PROBLEM

Service A charges a \$0.90 session fee plus \$0.11 per request. Service B charges \$0.11 per session plus \$0.90 per request. For five requests in one session, which is cheaper and by how much?

1 Model the total

Total = session fee + 5 × request price.

2 Evaluate both choices

$$A: 0.90 + 5 \times 0.11 = \$1.45$$

$$B: 0.11 + 5 \times 0.90 = \$4.61$$

3 Compare and verify

$$B - A = 4.61 - 1.45 = \$3.16.$$

$$\text{Check: } (5 - 1) \times (0.90 - 0.11) = 3.16.$$

Reasoning Data Coverage

Mixture component	Learning signal	Why it matters
Math and logic problems	Checked answers; worked solutions	Multi-step correctness
Programming tasks	Tests, execution, and good solutions	Behavior with executable feedback
Instruction / format tasks	Rules and labeled responses	Useful, controllable outputs
General conversations	High-quality ordinary responses	Retain helpful everyday behavior

Reasoning SFT Sample

PROMPT

3 boxes contain 8 pencils each. You give away 5 pencils. How many remain?
Use Final: <number>.

CHECKED ASSISTANT TARGET

$3 \times 8 = 24$ pencils.
 $24 - 5 = 19$ pencils.
Final: 19

Teacher forcing

Predict each target response token from the prompt and target prefix.

Loss on the worked solution and final answer. Prompt loss is masked.

Reasoning SFT Loss

$$\mathcal{L}_{\text{reason SFT}} = - \underbrace{\frac{1}{\sum_{b,t} m_{b,t}}}_{3} \sum_{b,t} \underbrace{m_{b,t}}_1 \underbrace{\log \pi_{\theta}(y_{b,t}^* \mid x_b, y_{b,<t}^*)}_2$$

1 Selected response

Include checked solution and answer targets; mask prompt and padding.

2 Next-token imitation

Fit the student to the supplied steps and answer under teacher forcing.

3 Mean selected tokens

Long responses contribute more targets under this batch-token average.

Reinforcement Learning with Verifiable Rewards

PROMPT

3 boxes contain 8 pencils each. You give away 5 pencils. How many remain?
Use Final: <number>.

POLICY-SAMPLED ANSWER

“Final: 19”



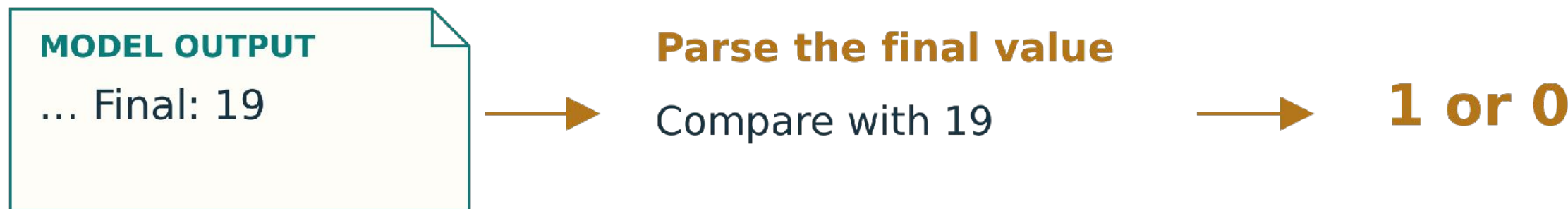
Check against hidden 19
Reward = 1

Another sampled answer: “Final: 29” gets reward 0.

Train from scored attempts. No gold solution text is supplied to the policy.

Verifier

A procedure that turns an answer into a score



Rule or test

Exact answer check, compiler, or unit tests

Model-based judge

A learned scorer can also output a reward.

Following example: fixed code, binary reward, no trainable verifier.

Group Relative Policy Optimization (GRPO)

Try the same problem several times. Score the answers, then favor those that did better than their group.

1 Sample

Several answers to one prompt



2 Compare

Each score against the group



3 Update

Clipped policy updates

The group supplies the baseline. No learned critic is needed.

GRPO Components



Old probabilities Stored from the policy that sampled this group

Reference Optional frozen anchor for a KL term

GRPO Reward Group

One prompt, four sampled answers. Correct final value: 19.

Answer	Parsed final value	Reward	Group advantage
1	29	0	-1
2	19	1	+1
3	19	1	+1
4	15	0	-1

Group-relative Advantage

$$A_i = \frac{R_i - \text{mean}(R_1, \dots, R_n)}{\text{std}(R_1, \dots, R_n) + \epsilon_{\text{num}}}$$

R_i Score for answer i in this prompt's group

Mean and scale Subtract the mean. Divide by the reward spread.

ϵ_{num} A tiny stabilizer if the group has zero spread

Rewards [0, 1, 1, 0] give advantages $\approx [-1, +1, +1, -1]$.

Use answer i 's advantage at every generated token in that answer.

GRPO Loss

Reuse PPO's clipped policy loss with the group advantage A_i .

$$\mathcal{L}_{\text{policy}}^{(i)} = -\frac{1}{T_i} \sum_t \min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i)$$

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\text{policy}}^{(i)}$$

Indices

i: answer, t: token, n: number of answers

Length T_i

Number of generated tokens in answer i

GRPO Reference Regularization

An optional reference term can be added to the policy loss.

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{n} \sum_i \left[\mathcal{L}_{\text{policy}}^{(i)} + \beta \frac{1}{T_i} \sum_t D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}} \mid s_{i,t}) \right]$$

Policy term

Favors above-average sampled solutions.

Reference term

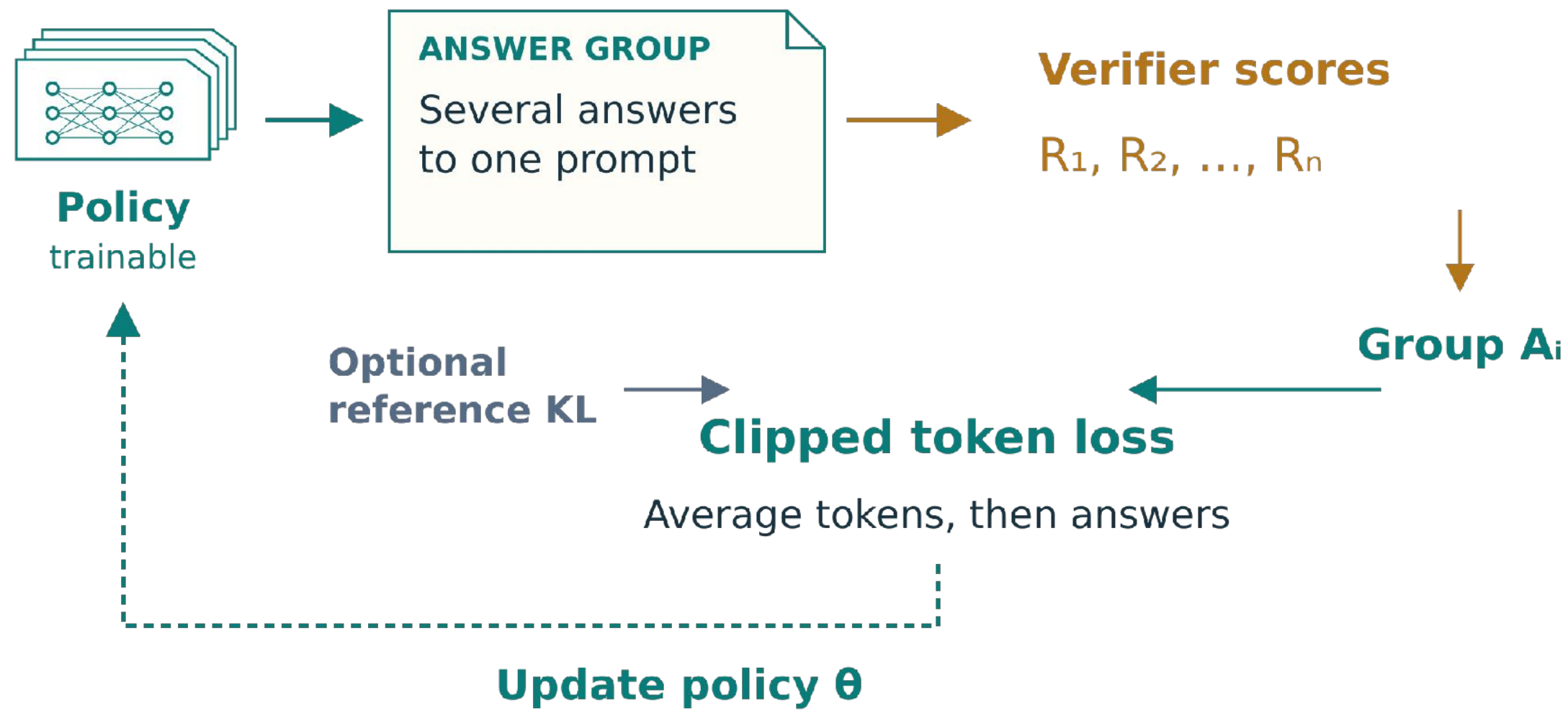
Discourages distribution drift at each prefix.

$\beta = 0$

Removes this optional reference contribution.

Here KL enters the loss directly. Group rewards come from the task checker.

GRPO Loss Flow

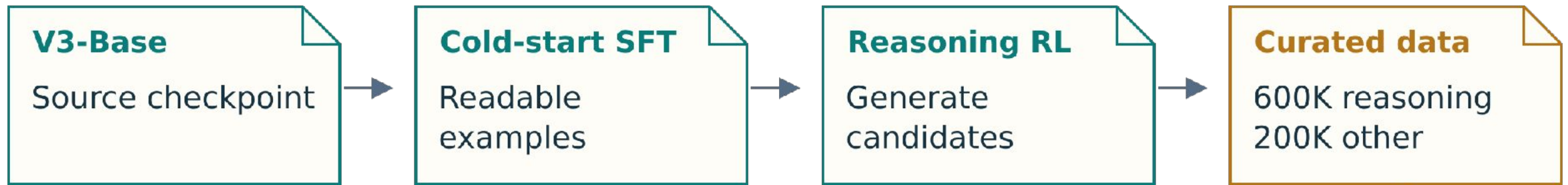


Verifier Limitations

Reward design	Benefit	Blind spots
Exact final-answer match	Cheap and reproducible	Parsing errors; equivalent forms
Code unit tests	Checks executable behavior	Missing cases; hard-coded outputs
Format compliance	Machine-readable responses	Correct format, wrong content
Step / process feedback	More local guidance	Costly labels; unreliable step scores

DeepSeek-R1 Training Stages

Generate a training set



Restart from V3-Base



Reasoning Cost Example

Setup	Token count	Interpretation
SFT: one 1,024-token target	$10,000 \times 1,024 = 10.24\text{M}$	Teacher generation may be earlier
RL: eight 1,024-token rollouts	$10,000 \times 8 \times 1,024 = 81.92\text{M}$	Sampling before policy updates
Extra work	Verifier, reference, and updates	Plus prompt tokens and overhead

Reasoning Generalization

USER REQUEST

"Which is greater: -0.11 or -0.9 ? Explain briefly."

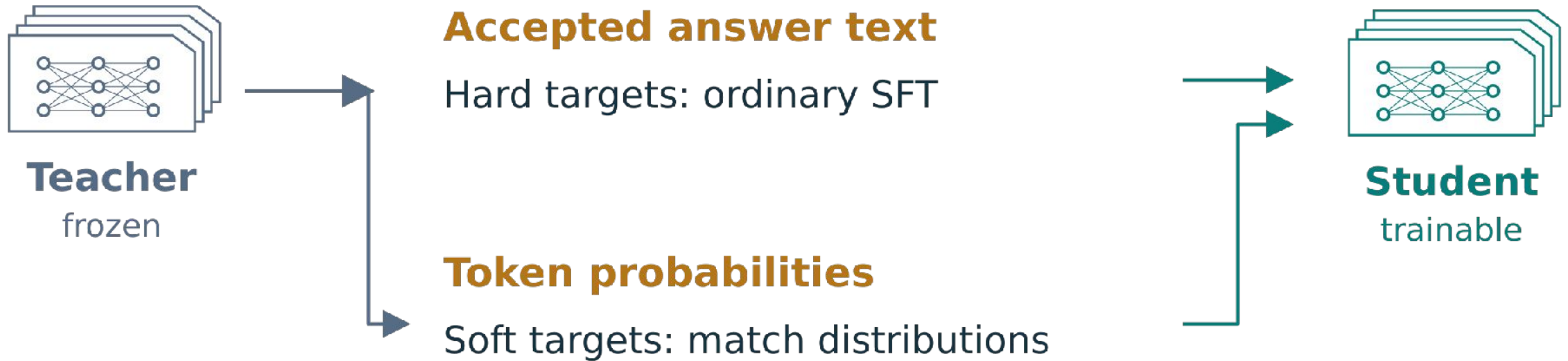
A memorized positive-number rule

" -0.9 is greater because 0.90 is greater than 0.11 ."

Desired generalized answer

" -0.11 is greater. For negative numbers, the value closer to zero is greater."

Knowledge Distillation



Distillation Temperature

$$p_i^{(\tau)} = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}} \quad \tau = \text{temperature}$$

Same logits [4, 2, 0]

$\tau = 1$



$\tau = 2$



Distillation Loss

$$\mathcal{L}_{\text{soft}} = \underbrace{\tau^2}_{\text{gradient scale}} \underbrace{D_{\text{KL}}(p_{\text{teacher}}^{(\tau)} \| p_{\text{student}}^{(\tau)})}_{\text{distribution matching}}$$

Without τ^2 , the student-logit gradient is

$$\frac{\partial D_{\text{KL}}}{\partial z_{\text{student},i}} = \frac{p_{\text{student},i}^{(\tau)} - p_{\text{teacher},i}^{(\tau)}}{\tau}$$

At high temperature, the probability gap also shrinks roughly as $1/\tau$.

Multiplying by τ^2 roughly offsets the resulting $1/\tau^2$ gradient shrinkage.

Agent Training



SFT: imitate action targets. RL: reward task success and useful tool decisions.

Tool-use Example

USER: "Can I refund order 472? It arrived 12 days ago."

1 Policy action

lookup_refund_policy()
lookup_order(472)

2 Tool observation

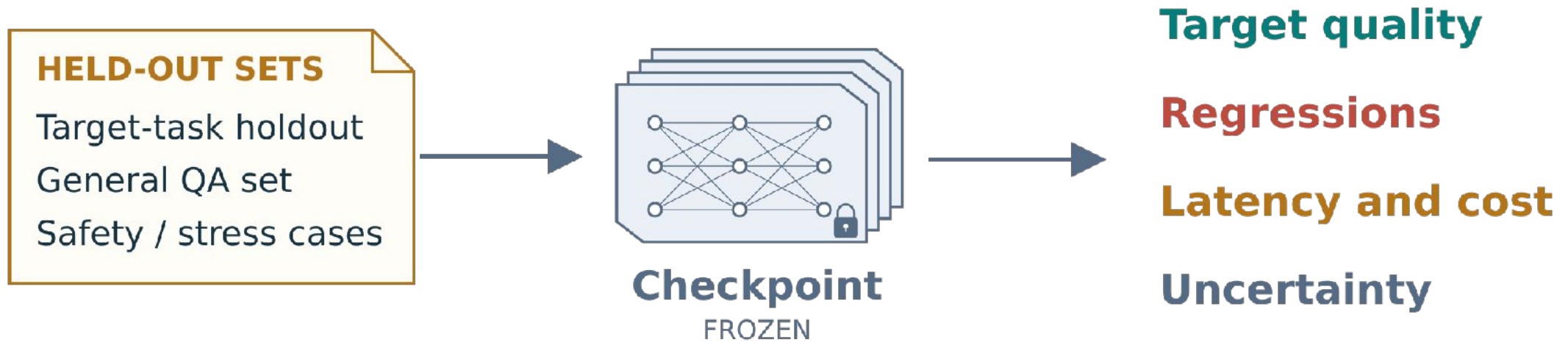
Window: 30 days
Opened item: false
Excluded category: false

3 Policy answer

"Yes. This order meets the 30-day and product eligibility rules."

Training trajectories label policy actions; the tool supplies observations.

Evaluation



No optimizer step. Fix prompting, sampling, and scoring.

Evaluation Metrics

Question	Representative evaluation	Metric or judgment
Predict unseen text?	Held-out language-model corpus	Cross-entropy / perplexity
Follow instructions?	IFEval; custom schema tasks	Constraint and format success
Reason or write code?	Math problems; HumanEval-style tests	Accuracy; executable pass rate
Give better answers?	Blinded human / rubric comparisons	Win rate with uncertainty
Work as a product?	Representative end-to-end tasks	Success, latency, cost, regressions

Perplexity

$$\text{CE} = - \underbrace{\frac{1}{M} \sum_t \log p_\theta(x_t \mid x_{<t})}_1 \quad \text{PPL} = \underbrace{\exp(\text{CE})}_2$$

1 Held-out token loss

Score observed tokens without updating model weights.

2 Exponential scale

For CE = 0.70 nats, perplexity is approximately 2.01.

Illustrative Evaluation Results

Checkpoint	Task accuracy	Valid JSON	General QA
Before task SFT	70%	40%	60%
After task SFT	90%	98%	52%
Change	+20 points	+58 points	−8 points

Pass@k

$$\widehat{\text{pass@}k} = 1 - \underbrace{\binom{n-c}{k}}_1 / \underbrace{\binom{n}{k}}_2$$

1 All-failing subsets

n sampled answers contain c passing answers. Choose k only from failures.

2 All possible subsets

The ratio is the probability that a subset of k has no passing answer.

Judge-based Evaluation

Possible bias	Why the score can mislead	Useful control
Position	The first or second answer is favored	Swap order and aggregate
Length / style	Polished verbosity looks higher quality	Apply a specific correctness rubric
Judge limitations	The judge cannot solve the task reliably	Use verifiers and expert checks
Shared blind spots	Judge and policy share systematic errors	Independent evaluation sources

Evaluation Reproducibility

Record	Why it matters
Checkpoint and dataset version	Makes the experiment identifiable
Prompt, chat template, and few-shot examples	Changes the model's effective input
Temperature, length limit, samples per task	Changes inference compute and output distribution
Splits, contamination checks, scoring code	Determines whether success is meaningful
GPU-hours, peak memory, latency, token counts	Shows whether the gain is worth the cost

Training Methods

Method	Target / problem	Data supplied
In-context	Specify behavior for this request	Instructions and examples in context
Pre-training	Learn a broad text distribution	Large curated text mixture
Mid-training	Deepen domain or context capability	Focused text + retention mixture
SFT	Demonstrate desired responses	Prompt–response examples
Distillation	Transfer teacher behavior	Teacher outputs or distributions

Training Methods (cont.)

Method	Target / problem	Data and learning signal
Reward modeling	Learn which responses are preferred	Ranked responses → scalar scores
PPO-based RLHF	Improve policy under a learned reward	Prompts, rollouts, reward model
DPO	Prefer chosen over rejected responses	Stored response pairs + reference
Reasoning SFT	Demonstrate useful solution patterns	Checked worked solutions
RLVR / agent RL	Improve checked task outcomes	Prompts, rollouts, verifier / environment

Training and Deployment Components

Phase	Trainable	Frozen / external	Deploy
Pre-training / SFT	PT: all weights SFT: all or adapters	Dataset	Policy
Reward-model fit	Scorer backbone + head	Preference labels	Usually used for training
PPO-based RLHF	Actor and critic	Reward and reference	Actor
DPO	Policy weights or adapters	Reference and pairs	Policy
GRPO with verifier	Policy weights or adapters	Verifier; optional reference	Policy

Training Cost

Method	Data budget	Compute drivers
Pre- / mid-training	Training tokens $\times$ passes	Model size, context, full optimizer state
SFT / distillation	QA pairs $\times$ sequence length $\times$ epochs	Trainable state; teacher-generation cost
Reward modeling	Prompts $\times$ compared responses	Scorer size; annotation and scoring
DPO	Pairs $\times$ both response lengths	Policy backward pass; reference scoring
PPO / GRPO	Prompts $\times$ rollouts $\times$ rollout length	Generation, scorer / critic, update passes

Training Budgets

Reference point	Quantity	How to use it
Llama 3.1 8B	~15T pre-training tokens 1.46M H100 GPU-hours	Scale reference; model training total
Alpaca 7B SFT	52K examples; 8 A100 × ~3 h	Historical tuning example
QLoRA 65B	One 48 GB GPU in the paper	A memory feasibility result
Our RL arithmetic	10K prompts × 8 × 1,024 = 81.92M generated tokens	Planning example, not a GPU forecast

Method Selection

- | | | | |
|----------|---|---|-------------------------------------|
| 1 | Missing instructions or current evidence | → | Prompting / retrieval |
| 2 | Repeated task / format errors | → | SFT, often with adapters |
| 3 | Poor preference or verifiable-task performance | → | Preference / reward training |

Lecture Summary

Input

What information is the model conditioned on?
Prompts, demonstrations, retrieved evidence.

Learning signal

What counts as a better output?
Observed tokens, preferences, or rewards.

Parameters

Which tensors receive optimizer updates?
All weights, selected layers, adapters, or none.